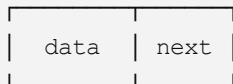


Address-based exercise set for both **Circular Singly Linked List (CSLL)** and **Circular Doubly Linked List (CDLL)**. The idea is that students are given **node addresses, data values, and logical order**, and they must **draw the linked-list figure and determine the pointer values**.

A. Circular Singly Linked List (CSLL)

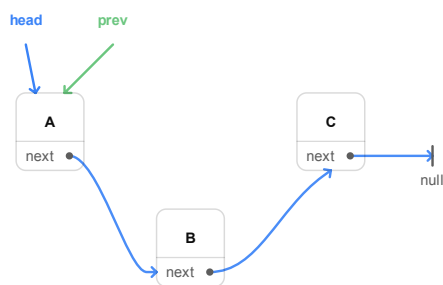
Node representation

Each node contains:



In a circular singly linked list:

- **HEAD** points to the first node.
- The last node's **next** points back to **HEAD**.
- There is **no NULL at the end**.



prev → A

Step 1 of 4

Delete B: Locate the previous node.

Insert

Delete

Step
p

1

[Give feedback](#)

Problem 1 — Construct a Circular Singly Linked List

Given:

Address	Data
1000	10
1050	20
1100	30
1150	40
1200	50

HEAD = 1000

Logical order:

1000 → 1050 → 1100 → 1150 → 1200 → HEAD

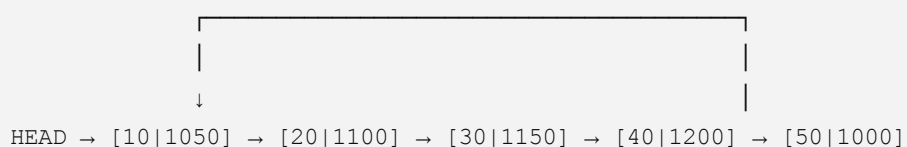
Task

1. Draw the circular singly linked list.
2. Fill the `next` field.
3. Show where the last node points.

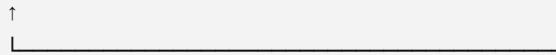
Solution

Address	Data	Next
1000	10	1050
1050	20	1100
1100	30	1150
1150	40	1200
1200	50	1000

Figure:



1000 1050 1100 1150 1200



Problem 2 — Random Memory Addresses

Given:

Address	Data
3000	30
1500	10
4500	50
2750	20
6000	40

HEAD = 1500

Logical order:

1500 → 2750 → 3000 → 6000 → 4500 → HEAD

Task

Complete the table and draw the circular list.

Solution

Address	Data	Next
1500	10	2750
2750	20	3000
3000	30	6000
6000	40	4500
4500	50	1500

Figure:

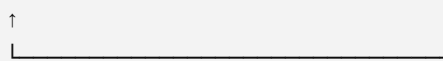
HEAD

↓

[10] → [20] → [30] → [40] → [50]

1500 2750 3000 6000 4500

↑



Teaching point: The largest address is not necessarily the last node.

Problem 3 — Delete from Beginning

Given:

Address	Data
5000	10
5050	20
5100	30
5150	40
5200	50

HEAD = 5000

Logical order:

5000 → 5050 → 5100 → 5150 → 5200 → HEAD

Task

Delete the first node.

Draw the list before and after deletion.

Solution

Before:

HEAD

↓

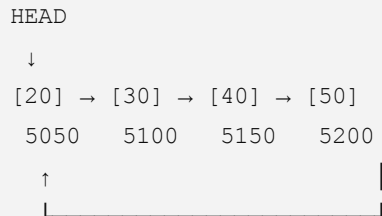
[10] → [20] → [30] → [40] → [50]

5000 5050 5100 5150 5200

Delete 5000 .

New HEAD = 5050 .

After:



Updated links:

```
HEAD = 5050
5200.next = 5050
```

Problem 4 — Delete from End

Given:

Address	Data
6000	11
6050	22
6100	33
6150	44
6200	55

Logical order:

6000 → 6050 → 6100 → 6150 → 6200 → HEAD

Task

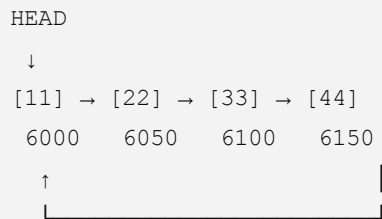
Delete the last node.

Solution

Last node = 6200 .

The node before it is 6150 .

After deletion:



Updated:

6150.next = HEAD

Problem 5 — Delete After a Given Node

Given:

Address	Data
7000	10
7050	20
7100	30
7150	40
7200	50

Logical order:

7000 → 7050 → 7100 → 7150 → 7200 → HEAD

Task

Delete the node **after address 7050**.

Solution

Node after 7050 = 7100 .

Before:

[10] → [20] → [30] → [40] → [50]
7000 7050 7100 7150 7200
↑
└────────────────────────┘

After:

[10] → [20] → [40] → [50]
7000 7050 7150 7200
↑
└────────────────────────┘

Updated:

7050.next = 7150

Problem 6 — Delete Before a Given Node

Given:

Address	Data
8000	10
8050	20
8100	30
8150	40
8200	50

Logical order:

8000 → 8050 → 8100 → 8150 → 8200 → HEAD

Task

Delete the node **before address 8150**.

Solution

Node before 8150 = 8100 .

Because this is a **singly** linked list, we need to find the node before 8100 , which is 8050 .

After deletion:

```
[10] → [20] → [40] → [50]
8000   8050   8150   8200
      ↑
      └──────────┘
```

Updated:

```
8050.next = 8150
```

Problem 7 — Insert at Beginning

Given:

Address	Data
9000	20
9050	30
9100	40
9150	50

New node:

Address	Data
8950	10

Task

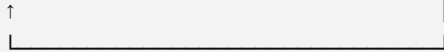
Insert node **8950** at the beginning.

Solution

Final order:

8950 → 9000 → 9050 → 9100 → 9150 → HEAD

```
HEAD
↓
[10] → [20] → [30] → [40] → [50]
8950   9000   9050   9100   9150
```



Important:

```
newNode.next = HEAD
lastNode.next = newNode
HEAD = newNode
```

Problem 8 — Insert After a Given Node

Given:

Address	Data
10000	10
10100	20
10200	40
10300	50

New node:

Address	Data
10400	30

Task

Insert 10400 after node 10100 .

Solution

Final order:

10000 → 10100 → 10400 → 10200 → 10300 → HEAD

```
[10] → [20] → [30] → [40] → [50]
10000 10100 10400 10200 10300
↑
└──────────────────────────┘
```

Updated:

```
10400.next = 10200
10100.next = 10400
```

B. Circular Doubly Linked List (CDLL)

Node representation

Each node has three fields:



In a circular doubly linked list:

```
HEAD->prev = TAIL
TAIL->next = HEAD
```

There is **no** `NULL` at either end.

Problem 9 — Construct a Circular Doubly Linked List

Given:

Address	Data
2000	10
2050	20
2100	30
2150	40
2200	50

```
HEAD = 2000
```

Logical order:

2000 → 2050 → 2100 → 2150 → 2200 → HEAD

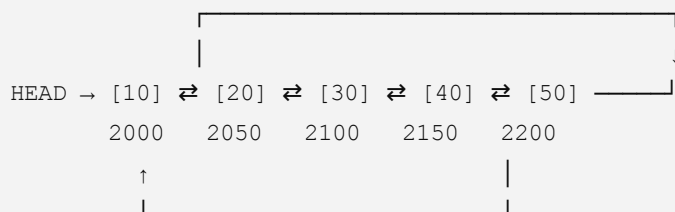
Task

Complete `prev` and `next` .

Solution

Address	Prev	Data	Next
2000	2200	10	2050
2050	2000	20	2100
2100	2050	30	2150
2150	2100	40	2200
2200	2150	50	2000

Figure:



The important circular links are:

```
2000.prev = 2200
2200.next = 2000
```

Problem 10 — CDLL: Delete from Beginning

Given:

Address	Data
3000	10
3050	20
3100	30
3150	40

Address	Data
3200	50

HEAD = 3000

Task

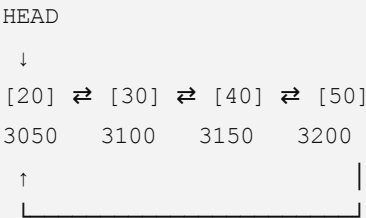
Delete the first node.

Solution

Delete 3000 .

New HEAD = 3050 .

Final list:



Final links:

Address	Prev	Data	Next
3050	3200	20	3100
3100	3050	30	3150
3150	3100	40	3200
3200	3150	50	3050

Problem 11 — CDLL: Delete from End

Given:

Address	Data
4000	11
4050	22

Address	Data
5050	20
5100	30
5150	40
5200	50

Task

Delete the node **after** 5050 .

Solution

Node after 5050 = 5100 .

Final:

```
[10] ⇌ [20] ⇌ [40] ⇌ [50]
5000   5050   5150   5200
  ↑           |
  └──────────┘
```

Updated links:

```
5050.next = 5150
5150.prev = 5050
```

Problem 13 — CDLL: Delete Before a Given Node

Given:

Address	Data
6000	10
6050	20
6100	30
6150	40

Address	Data
6200	50

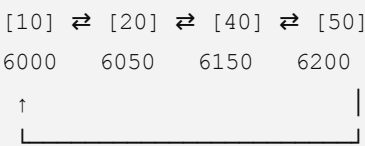
Task

Delete the node **before** 6150 .

Solution

Node before 6150 = 6100 .

Final:



Updated:

```
6050.next = 6150
6150.prev = 6050
```

Problem 14 — CDLL: Insert at Beginning

Given:

Address	Data
7000	20
7050	30
7100	40
7150	50

New node:

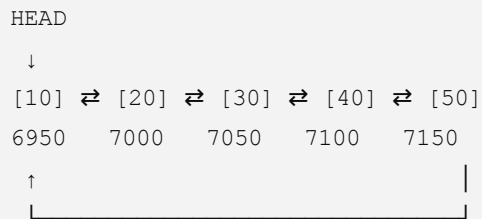
Address	Data
6950	10

Task

Insert 6950 at the beginning.

Solution

Final:



Important links:

```
newNode.prev = TAIL
newNode.next = old HEAD

TAIL.next = newNode
old HEAD.prev = newNode

HEAD = newNode
```

Problem 15 — CDLL: Insert After a Given Node

Given:

Address	Data
8000	10
8050	20
8100	40
8150	50

New node:

Address	Data
8200	30

Task

Insert 8200 after 8050 .

Solution

Final order:

8000 → 8050 → 8200 → 8100 → 8150

```
[10] ⇌ [20] ⇌ [30] ⇌ [40] ⇌ [50]
8000  8050  8200  8100  8150
  ↑
  └──────────────────┘
```

Updated links:

```
8200.prev = 8050
8200.next = 8100

8050.next = 8200
8100.prev = 8200
```

Problem 16 — CDLL Challenge: Random Addresses

Given:

Address	Data
9100	70
8500	20
7800	50
9500	90
8200	30

HEAD = 8500

Logical order:

8500 → 8200 → 7800 → 9100 → 9500 → HEAD

Task

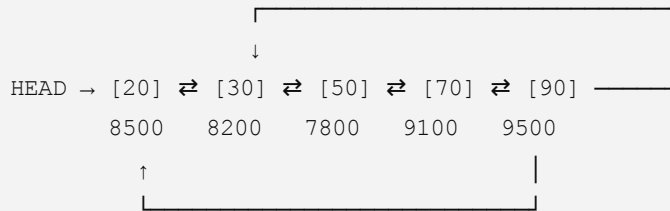
1. Draw the CDLL.

2. Complete `prev` and `next` .
3. Identify `HEAD` and `TAIL` .
4. Give the forward traversal.
5. Give the backward traversal.
6. Delete the node after `7800` .

Solution

Address	Prev	Data	Next
8500	9500	20	8200
8200	8500	30	7800
7800	8200	50	9100
9100	7800	70	9500
9500	9100	90	8500

Figure:



Forward:

20 → 30 → 50 → 70 → 90 → 20 ...

Backward:

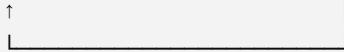
90 → 70 → 50 → 30 → 20 → 90 ...

Delete node after `7800` :

Node after 7800 = 9100

Final:

[20] ⇌ [30] ⇌ [50] ⇌ [90]
8500 8200 7800 9500



Quick Comparison for Students

Operation	Singly Linked	Circular Singly	Doubly	Circular Doubly
Delete beginning	Easy	Easy, update last link	Easy	Easy
Delete end	Traverse to previous	Traverse to previous	Easy with <code>prev</code>	Easy with <code>prev</code>
Delete after	Easy	Easy	Easy	Easy
Delete before	Need traversal	Need traversal	Direct using <code>prev</code>	Direct using <code>prev</code>
Last node points to	<code>NULL</code>	<code>HEAD</code>	<code>NULL</code>	<code>HEAD</code>
First node <code>prev</code>	—	—	<code>NULL</code>	<code>TAIL</code>
Forward traversal ends	<code>NULL</code>	At <code>HEAD</code> again	<code>NULL</code>	At <code>HEAD</code> again
Backward traversal	Not possible	Not possible	Possible	Possible

Recommended lab sequence

For students, I would arrange the exercises in this order:

Singly Linked List

1. Create list
2. Insert beginning
3. Insert end
4. Insert after
5. Insert before
6. Delete beginning
7. Delete end

8. Delete after
9. Delete before
10. Random-address challenge

Circular Singly Linked List

1. Create circular list
2. Insert beginning
3. Insert end
4. Insert after
5. Delete beginning
6. Delete end
7. Delete after
8. Delete before

Circular Doubly Linked List

1. Create CDLL
2. Forward/backward traversal
3. Insert beginning
4. Insert end
5. Insert after
6. Insert before
7. Delete beginning
8. Delete end
9. Delete after
10. Delete before
11. Random-address challenge

This progression lets students see exactly **what changes when** `NULL` **is replaced by** `HEAD` **, and when the** `prev` **pointer is added.**